Creating and Sharing Genomic Annotation Modules with AI: The Just-DNA Ecosystem

Anonymous submission to EASRP 2026

Abstract

Genome analysis is an iterative workflow—ask, select data, score, annotate, inspect, and revise—and it is increasingly done inside AI coding assistants rather than fixed graphical interfaces. But general-purpose language models are unreliable when used directly for genomics: they fabricate variant identifiers, misassign effect directions, and attach real PMIDs to the wrong paper. The Just-DNA ecosystem redirects these assistants from generating disposable code to producing structured, versioned data: *genomic annotation modules* where every variant is linked to its genotype, evidence, effect size, and a verbatim passage from the source paper. The output is an editable data artifact, not generated analysis code, and a domain expert can review it in a spreadsheet without specialized tools.

This paper presents the plugin ecosystem that makes this possible. The authoring plugin, **just-module-creator**, runs inside the AI coding assistants that bioinformaticians and developers already use daily, so module authoring requires no new infrastructure. The underlying toolchain validates authored files against a live schema, checks identifiers against reference databases, and compiles modules into a portable artifact. A shared registry lets independent authors publish, version, and exchange modules—the package-registry pattern applied to genomic annotation knowledge. Once installed, a module can be applied to a personal genome within minutes; the annotation consumer, Just-DNA-Lite, is the subject of a companion paper [Anonymous, 2026]. The software is open-source and available at `https://anonymous.4open.science/r/just-dna-registry` (anonymized for review).

1 Introduction

Genomic annotation connects variant calls to the knowledge that gives them meaning: trait associations, clinical classifications, effect sizes, and the evidence behind each claim. This knowledge is scattered across papers, databases, and preprints, and it changes as new studies appear. Turning it into something a computational pipeline can use requires structured, validated records with unambiguous variant identifiers, genome-build coordinates, and traceable citations.

Researchers increasingly use AI coding assistants such as Claude Code and Codex for this kind of work. These tools are powerful, but when asked to analyze variants they typically generate a script: code that may fabricate identifiers [Jin et al., 2024b], attach a real PMID to the wrong paper [Ji et al., 2023, Gao et al., 2023], reverse an allele, or bypass established filters. Each session may produce slightly different code, and the result must be reviewed for correctness, efficiency, and security before it can be trusted. The output is a one-off answer tied to one session, not a reusable resource.

This paper presents the Just-DNA ecosystem, which gives these same assistants a different job. Instead of generating code, the assistant produces structured data: a *module* of editable CSV tables where each variant is linked to its studies, citations, effect sizes, and supporting passages. The input can be anything the assistant can read, from a single sentence describing a trait to a collection of papers or a summary from another model. Every variant is verified against reference databases such as dbSNP; dedicated checking tools validate gene symbols against HGNC, trait identifiers against ontology services, and flag decisions that require domain expertise. The result contains no executable code. Once compiled into Parquet, a columnar format optimized for fast queries, Just-DNA-Lite joins the module to a personal genome and returns annotated results in under a minute [Anonymous, 2026]. Modules are versioned and shareable: they can be kept locally, rehearsed in a staging registry, or published for others to install and use. The same files can also be written or edited entirely by hand.

This paper makes four contributions:

1. It describes the module contract shared by manual editors, AI-assisted authoring tools, versioned stores, and the genome-analysis consumer.
2. It presents `just-module-creator`, an authoring plugin that exposes the public capabilities of `just-dna-format`, `just-dna-compiler`, `just-dna-enricher`, and `just-dna-registry` as typed Model Context Protocol (MCP) tools [Anthropic, 2024]. The plugin reads the installed schema instead of carrying a copy.
3. It defines how modules move between a local store, a deletable test registry, and the production registry while preserving provenance and decisions that still require review.
4. It proposes an evaluation protocol for variant recovery, citation identity, and effect-direction agreement, and reports catalog statistics from the first eight published modules across five independent namespaces.

2 Related work

Genome annotation and catalogs. A variant call records what differs between a sample and a reference genome. Annotation adds information about that call, such as its predicted consequence, known clinical classification, reported trait association, or the conclusion assigned to a particular genotype. VEP, ANNOVAR, and OpenCRAVAT/OakVar perform this kind of lookup by joining a callset to established resources [McLaren et al., 2016, Wang et al., 2010, Pagel et al., 2020]. ClinVar and the GWAS Catalog publish records that can supply annotation content [Landrum et al., 2018, Sollis et al., 2023]. HGMD derives variant–disease associations from the literature through manual review, though the public version updates infrequently and does not include explicit pathogenicity calls [Stenson et al., 2020]. ClinGen provides expert variant curation within the ACMG/AMP framework [Rehm et al., 2015, Richards et al., 2015]. PharmGKB and CPIC supply pharmacogenomic annotations including diplotype-level clinical recommendations [Whirl-Carrillo et al., 2012]. The GA4GH Genomic Knowledge Standards define interoperable representations for variant annotations [Global Alliance for Genomics and Health, 2022]. The problem addressed here begins before the join: selected evidence must be turned into a reviewable, machine-checkable collection that can be distributed and applied consistently.

Table 1 narrows the comparison to systems adjacent to the authoring task: extension-based annotation platforms, structured literature extraction, unconstrained model drafting, and `just-module-creator`.

Biomedical text mining and variant extraction. Conventional named entity recognition systems such as tmVar3 and PubTator3 have improved gene and variant tagging in biomedical text [Wei et al., 2022, 2024]. LitVar 2 links literature paragraphs with variant records from external databases, accelerating knowledge retrieval [Allot et al., 2023]. However, these tools are largely limited to entity-level extraction and do not recover relational information such as variant-specific pathogenicity across disease contexts or the experimental evidence supporting a classification. PubMind applies instruction-tuned LLMs to extract variant–disease–pathogenicity associations directly from over 41 million PubMed abstracts and 5.4 million full-text articles, producing a database of approximately 1.3 million unique variants with contextual annotations [Wang and Wang, 2026]. These systems demonstrate that LLM-based literature mining can recover structured variant knowledge at scale, but their output is a flat database rather than a versioned, schema-validated module that an annotation consumer can install and apply to a genome.

Table 1: Qualitative comparison of systems adjacent to annotation authoring. The rows have different primary tasks; the capability columns compare how their structured outputs are created, checked, and shared. Runtime annotation speed is outside this comparison and is covered in the companion platform paper [Anonymous, 2026].

System	Kind / primary task	*Versioned modules*	*Schema validation*	*Provenance tracking*	*AI-assisted authoring*	*Shared catalog*
OpenCRAVAT / OakVar	annotation platform / user-supplied annotation extensions	✓	✓	–	–	✓
PubMind-DB	text-mined database / LLM extraction of variant–disease–pathogenicity records	–	✓	✓	✓	✓
Raw LLM	general-purpose model / unconstrained drafting	–	–	–	✓	–
just-module-creator	authoring plugin / create, review, and publish installable modules	✓	✓	✓	✓	✓

Criteria. Versioned modules: user-installable or distributable annotation units with explicit versions. *Schema validation*: structured records are checked against a declared machine-readable schema. *Provenance tracking*: record- or module-level source links travel with the output. *AI-assisted authoring*: a language model participates directly in creating or revising the structured output. *Shared catalog*: outputs or extensions are discoverable through a shared service. A dash means the capability is not part of the system's core public workflow, not that no related function can be added around it.

Factuality and tool use. Language models can produce fluent text around a false identifier or citation [Ji et al., 2023]. Benchmarks on genomic question-answering show that models hallucinate gene names, variant identifiers, and functional annotations [Jin et al., 2024b]. Retrieval-augmented approaches and tool use reduce the need to improvise when a model can call a typed operation instead [Schick et al., 2023, Jin et al., 2024a]. The Model Context Protocol provides a standard for connecting models to external tools [Anthropic, 2024]. Biomedical MCP services can search literature, ontologies, and variant databases. just-module-creator uses the same general approach, then carries the result into module files that the rest of the Just-DNA pipeline can inspect and reuse.

Agent orchestration. Many scientific-agent systems assign research and review to several model instances [Hong et al., 2023]. The plugin described here does not require its own orchestration runtime. Claude Code or Codex supplies the agent, while the plugin supplies the tools and the written workflow. This paper evaluates that authoring path, not a particular choice of language model or agent-team topology.

3 Method

3.1 The module artifact

At the artifact boundary, a just-dna annotation module is an editable directory. It always contains `module_spec.yaml` and at least one recognized table. The table records the thing being annotated, such as a genotype, a diplotype, or a measured quantity. Evidence and machine-produced sidecars are stored separately. Compilation converts this human-readable directory into machine-readable Parquet files and a content-addressed `manifest.json` [DNA Seq contributors, 2026]. The compiled artifact is what a consumer installs and joins to a genome.

The authored files are ordinary YAML, CSV, Markdown, and optional image files. A domain expert can open them in a spreadsheet to review and correct an AI draft without special

tools. just-module-creator operates on this same directory rather than introducing a separate format for AI-produced content. The module describes how a genotype or measurement should be interpreted if a consumer encounters it. Just-DNA-Lite, or another compatible consumer, supplies the genome or measurement later.

The author does not need to supply this directory, or any CSV file, at the start of a session. The conversation may begin with only a trait, a gene, or an idea. It may instead include a deep research report, paper, PDF, link, or custom file; request rows from a provider such as ClinVar, CPIC, or ClinPGx; or point to an existing module on disk or in the catalog. The `/create-module` router identifies the appropriate entry point. From an idea alone, the assistant can search the catalog and literature before selecting table kinds from the live schema. Scaffolding and row authoring then create the editable directory described above.

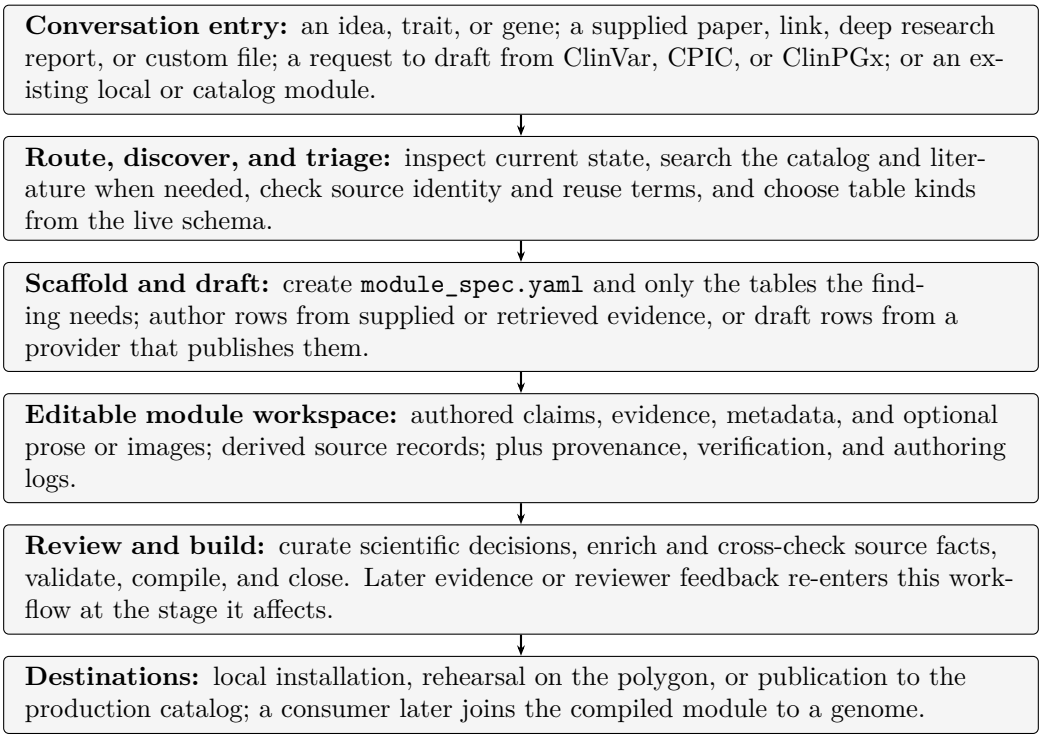

Figure 1: From conversation to a usable module. Module files are intermediate artifacts created by the workflow, not required user inputs. Authored claims remain distinct from derived source records inside the editable workspace.

The table kind follows the subject of the claim. For example, in format 0.6.6 used for this manuscript, a single-variant claim belongs in `variants.csv`; evidence for those rows belongs in `studies.csv`. Pharmacogenomic diplotypes, copy-number ranges, repeat counts, and published polygenic-score identifiers use other tables. The manuscript does not reproduce their column lists. The plugin calls `describe_table` and `table_requirements`, which read the installed pydantic models. An author therefore sees the schema accepted by the compiler running in that session.

3.2 Architecture and lifecycle

Module authoring and sample analysis remain separate until Just-DNA-Lite joins a compiled module to a local genome. On the authoring path, a person may edit the files directly or use an AI assistant to draft them. Both routes then pass through the same enrichment and compilation steps before the module enters a local or shared store. Figure 2 shows the two paths and the boundary between them.

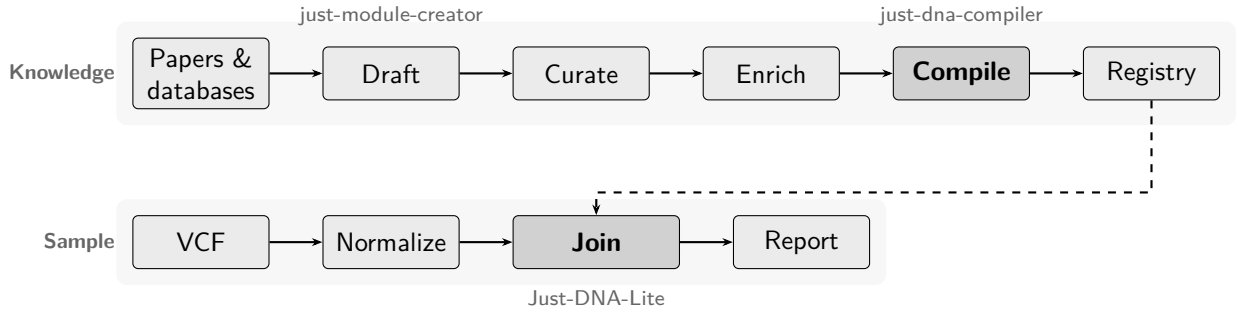

Figure 2: The two paths of the Just-DNA ecosystem. The knowledge path (top) produces a compiled module from sources; the sample path (bottom) joins that module to a local genome. The compiler is deterministic and offline. The dashed arrow marks the point where the two paths meet: a published or locally installed module is consumed by Just-DNA-Lite.

Table 2: The module lifecycle and the two sample-processing paths.

Path	Stages
Module lifecycle	papers and databases → manual editing or AI draft → enrichment and compilation → local store or registry
Sample annotation	VCF → normalization and quality filtering → joins with selected modules and reference data → annotated tables and report

On the sample path, normalization gives equivalent variants a consistent representation and computes the genotype used by later joins. Configured quality filters can remove calls that fail VCF filter status, depth, or quality criteria. The pipeline writes the normalized data to Parquet, so each selected module can use a streaming join instead of independently parsing the original VCF. The companion platform paper [Anonymous, 2026] describes the sample path in detail, including annotation speed benchmarks (a whole-genome VCF annotated in under 40 seconds) and polygenic risk score computation via `just-prs`.

Within `just-module-creator`, the language model participates only in the knowledge path. It can search and read sources, draft module rows, and help review disagreements. VCF filtering, module joins, and polygenic scoring remain ordinary software steps with explicit inputs and repeatable code.

No single package implements the whole path. Each component owns a boundary that another component can call without reimplementing its rules. Within the module toolchain, dependencies point inward: enricher → compiler → format. The format and compiler do not fetch remote data. The enricher performs source lookups and writes the derived sidecars used later by the compiler. The registry publishes the compiled result, and Just-DNA-Lite consumes it in a separate application.

Of these components, just-module-creator is the agent-facing authoring entry point examined most closely in this paper. It is distributed as an application whose public contract is the MCP tool surface and the accompanying workflow documents. The same source tree ships plugin manifests for Claude Code and Codex. Both hosts launch the same server and load the same skills.

3.3 Determinism, evidence, and responsibility

Within one compiler version, `just-dna-compiler` tests that a valid specification can be compiled, reversed, and compiled again without changing its authored content signature. just-

Table 3: Responsibilities across the Just-DNA ecosystem.

Component	Responsibility	Does not
`just-module-creator`	research and authoring workflow	process a genome
`just-dna-format`	schema and identity rules	access the network
`just-dna-compiler`	validate, compile, reverse, and sign	access the network
`just-dna-enricher`	resolve, draft, and cross-check	decide scientific meaning
`just-dna-registry`	publish, discover, and download modules	author module rows
Just-DNA-Lite	filter VCFs, join annotations, and produce reports	define module schema or compiler rules

module-creator calls that compiler and pins the resolution setting.[1] It does not implement a second compiler or present the upstream round-trip tests as evidence that an assistant extracted a paper correctly.

This separation avoids a circular evaluation. If an assistant writes a wrong claim in valid syntax, the compiler may preserve it perfectly. Reproducible output shows that the compiler kept the input stable; it does not show that the input agrees with the literature.

The workflow follows the module from an idea to one of its destinations:

`origin` → `scaffold` → `draft` → `curate` → `enrich` → `check` → `compile` → `close` → `rehearse` → `publish`

The origin determines where the work begins. A source such as ClinVar, CPIC, or ClinPGx may already publish rows, in which case a drafting tool can prepare a partial table. A paper or a free-text request instead begins with evidence search and manual authoring. Scaffolding creates the files and placeholders required for the selected table kind.

Curation follows drafting because a drafted row may still contain decisions that the source cannot make for the module: which genotype the claim concerns, how a weight should be interpreted, and how the conclusion should be worded. These are the decisions where domain expertise has the most leverage. The workflow presents them to the author rather than silently choosing values that merely satisfy the schema.

Enrichment resolves identifiers and writes sidecars such as `resolution.csv` and `literature.csv`. These files record the source answers used for later offline compilation. Compilation itself does not use the network.

After compilation, `close` records a statement in `verification.json` and binds it to the authored files. Editing those files invalidates the closure. In the current 0.x format a module can still compile and publish without a closure, but it carries a warning. Closing is an attributed declaration of completion. It does not prove that the module is correct.

Most modules return to this workflow. A later source release, a new paper, or a reviewer's correction begins a revision pass. The router inspects the existing directory so that the assistant does not treat a revision as a blank first draft and overwrite decisions whose history matters.

The compiler reports problems but does not edit authored values. just-module-creator is the authoring application, so its workflow permits edits. That permission comes with limits.

First, tool-mediated overrides append a record to `logs/authoring.log` and preserve a reason in `provenance.json`. A general hand edit is not captured automatically. The server therefore instructs an assistant to call `record_override` after such a change. This is a known gap between the desired provenance record and what the current tools can enforce.

[1] The plugin always calls the compiler with `resolve_with_ensembl=True`. The parameter name is misleading: setting it to false disables all resolution, including an injected `resolution.csv`, and the compiler can then succeed with null coordinates that cannot match a VCF.

Second, the workflow does not fill a value from the source that will later be used to check that same value. Such a check would compare the source with itself. The same problem appears when a row uses an article title as its `provenance_quote`: the title is guaranteed to occur in the article, so the resulting match provides no evidence that anyone located support for the row's claim.

Third, disagreement with an archive requires review of both sides. An archive may lag a retraction, a meta-analysis, or a later reclassification. Replacing the module's value with the archive's value can therefore make a module worse. `record_override` stores which field differs, what the source said, who made the decision, and why the authored value should remain. The record does not turn the disagreement into a passed check.

An assistant may read retrieved full text and locate a `provenance_quote`. It must copy the passage verbatim and identify who located it in `StudyRow.curator`. This attribution helps a reviewer decide where to look closely. It does not transfer responsibility away from the human author. The assistant can also follow a citation into its supplementary tables to locate per-variant statistics—effect sizes, p-values, and allele frequencies—that the main text often summarizes but does not reproduce row by row.

3.4 Local and shared module stores

In this paper, a module store is a place from which a compiled module can be installed or discovered. A local installation is useful during authoring. The registry provides two shared stores for publication rehearsal and release.

The registry has two independent instances. Production is the catalog used by consumers. The staging registry, called the polygon and selected with `target=test`, is a rehearsal environment where an author can delete a test publication. Writes default to the polygon. Catalog reads require an explicit target so that an author does not publish to one instance, read from the other, and mistake the missing result for a failed publication.

The production catalog is immutable. Publishing a version also claims its authored content by hash, and yanking the version does not release that claim. The workflow therefore rehearses on the polygon and asks separately before a production publish. The registry accepts the specification, runs its own enrichment and strict compilation, and stores the artifact it produced. It does not rely on a locally claimed digest, and it does not require the author to sign the local artifact. Inclusion in either catalog distributes a module; it is not scientific review, clinical validation, or endorsement.

Publication is not the only way to use a module. `/module-install-local` installs a compiled artifact into a Just-DNA-Lite checkout so it can be tested with a local VCF. This route tests the annotation connection between the module and the consumer. A polygon publication tests the separate connection between the module and the registry.

4 Results

4.1 Production catalog

At the time of writing, the production registry holds eight published modules across 19 versions and five independent namespaces (three distinct owners). Table 4 summarizes the published modules.

The modules span three independent namespaces from three owners. The largest module (`risk_impulsivity_snps`, 474 variants across 325 genes) and the smallest (`lactose_tolerance`, 2 variants in 1 gene) both compiled with strict resolution and fully resolved coordinates. The `big_five_personality_snps` module went through four published versions (1.0.0 through 2.1.0), illustrating the revision workflow: 1.0.0 was the initial GWAS Catalog extraction, 1.0.1 back-

Table 4: Modules published to the production registry as of August 2026. All modules target GRCh38 and were compiled with strict resolution.

Module	Variants	Studies	Genes	Versions
big_five_personality	330	390	185	4
risk_impulsivity	474	–	325	1
cognitive_intelligence	32	–	31	1
aggression_anger	28	–	22	1
bodybuilding	13	–	13	1
placebo_response (v2)	3	–	3	2
placebo_response_claude	3	–	3	1
lactose_tolerance	2	8	1	2
Total	**885**			**19 versions**

populated schema axes, 2.0.0 added polygenic score references, and 2.1.0 corrected the weight normalization.

Modules authored with `just-module-creator` carry `curator: ai-module-creator` in their manifest authorship records. Three modules were authored by people who are not the plugin's developers, indicating that the tool surface is usable beyond the original team.

4.2 Proposed evaluation protocol

The creation evaluation begins with an expert-checked fixture. Each fixture contains a free-text prompt and the `module_spec.yaml`, `variants.csv`, and `studies.csv` that the assistant should recover. The generated module is compared with the fixture on three questions:

- Variant recall is the fraction of ground-truth (*rsID*, *genotype*) rows present in the generated module.
- Citation identity requires every cited PMID to exist and to name the paper indicated by the title returned during search. Existence alone is insufficient.
- Weight-sign accuracy measures whether the generated effect direction agrees with the fixture. Magnitude is reported separately because a module weight is an authored choice and is not copied from a GWAS beta.

The development fixtures `fto_bmi` (one locus at rs1421085) and `longevity_2026` are available in the repository. A larger adjudicated set and repeated runs are needed to support a performance claim. We therefore report no recall or precision estimate in this paper and present the protocol as a framework for future evaluation.

4.3 How to read a green result

Several outputs look reassuring while answering a narrower question than a reader may expect. Strict compilation checks reproducibility and applies the compiler's blocking rules; it does not establish biological correctness. A digest match likewise shows that bytes or authored content agree under a specified comparison.

A source timeout produces an unknown result, not a negative one and not a pass. The tools preserve this distinction with null and unknown values. The title-as-quote observation above is another illustration: a green check can mean that the instrument could not have failed rather than that it found something.

The proposed evaluation protocol counts none of these as evidence of accurate extraction. Compiler round-trip behaviour is tested by `just-dna-compiler` and is not counted again as module creation accuracy.

5 Discussion

Genomic data is becoming personally accessible, and people already use AI assistants—coding tools such as Claude Code and Codex, or plain chat applications such as ChatGPT—to interpret their results [Counts, 2026]. Restricting this is not viable: regulatory frameworks add compliance burdens, yet users find workarounds or proceed informally. The more effective response is to build a community where the work happens in the open, where domain professionals can review what others publish, and where the tools enforce structure. A module is visible, reviewable, and correctable; a one-off chat session is none of these.

The safety concern is real but cuts deeper than AI. Candidate-gene studies have failed to replicate [Border et al., 2019], GWAS effect sizes routinely shrink in independent cohorts [Ioannidis, 2005], direct-to-consumer tests have produced false-positive rates above 40% [Tandy-Connor et al., 2018], and variant reclassifications have led to genetic misdiagnoses [Manrai et al., 2016]. A person from a more deterministic field can easily mistake a statistical association for a causal mechanism. The ecosystem is therefore explicitly research-only, and educating users about these limitations is as important as the tools. A module that faithfully reflects a study which later fails to replicate is not wrong on its own terms, but a user acting on it may be harmed all the same.

The registry turns individual effort into shared infrastructure: authors version, extend, correct, and republish modules—the pattern software engineering solved with package registries. The catalog already holds modules from independent authors, and the plugin's MCP tools are listed on biocontext.ai so other agent platforms can use them independently. Modules contain no individual-level data; Just-DNA-Lite applies them locally, so a person's VCF never leaves their machine and annotation runs raise no GDPR concerns. The knowledge travels through the registry; the genome does not.

Limitations. The module system launched in August 2026 and the catalog is less than a month old. We are developing mechanisms to involve genetic counselors and domain-expert agents in quality evaluation, and to communicate to citizen scientists that many modules reflect early-stage research rather than clinical-grade evidence. Just-DNA-Lite provides an extensive FAQ and science-literacy guide; the module catalog needs comparable guidance. The evaluation contains no creation accuracy estimate; the available fixtures are too small for a performance claim. The authoring workflow targets GRCh38 and does not perform liftover. Runtime benchmarks are in the companion paper [Anonymous, 2026].

6 Conclusion

People are already using AI assistants to interpret genomic data, and the volume of such use will only grow. The Just-DNA ecosystem channels that work into versioned, schema-validated modules that accumulate as shared infrastructure rather than evaporating with each chat session. The registry makes genomic annotation knowledge composable: an author publishes once, and others install, extend, and build on the result. Eight modules from five independent namespaces show that the path from first draft to published, reusable annotation is functional today.

The software is open-source and intended for research use only.

References

Alexis Allot, Kyubum Lee, Qingyu Chen, Ling Luo, and Zhiyong Lu. Tracking genetic variants in the biomedical literature using LitVar 2.0. *Nature Genetics*, 55:901–903, 2023. doi: 10.1038/s41588-023-01414-x.

Anonymous. Just-DNA-Lite: local genome annotation and polygenic risk scoring with deterministic modules, 2026. Companion platform paper (in preparation; citation anonymized for review).

Anthropic. Model context protocol. `https://modelcontextprotocol.io`, 2024. Specification and documentation.

Richard Border, Emma C. Johnson, Luke M. Evans, Andrew Smolen, Noah Berley, Patrick F. Sullivan, and Matthew C. Keller. No support for historical candidate gene or candidate gene-by-interaction hypotheses for major depression across multiple large samples. *American Journal of Psychiatry*, 176(5):376–387, 2019. doi: 10.1176/appi.ajp.2018.18070881.

Aisha Counts. An australian tech entrepreneur used AI to make a cancer vaccine for his dog. *Fortune*, March 2026. URL `https://fortune.com/2026/03/15/australian-tech-entrepreneur-ai-cancer-vaccine-dog-rosie-unsw-mrna/`. Accessed 2026-08-30.

DNA Seq contributors. just-dna-format: schema, compiler, and enricher for just-dna annotation modules. `https://anonymous.4open.science/r/just-dna-compiler`, 2026. Software. Packages `just-dna-format`, `just-dna-compiler`, `just-dna-enricher`.

Tianyu Gao, Howard Yen, Jiatong Yu, and Danqi Chen. Enabling large language models to generate text with citations. *arXiv preprint arXiv:2305.14627*, 2023. URL `https://arxiv.org/abs/2305.14627`.

Global Alliance for Genomics and Health. GA4GH genomic knowledge standards. `https://www.ga4gh.org/product/genomic-knowledge-standards/`, 2022. Interoperability standards for variant annotation and interpretation.

Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Zhong Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. *arXiv preprint arXiv:2308.00352*, 2023. URL `https://arxiv.org/abs/2308.00352`.

John P. A. Ioannidis. Why most published research findings are false. *PLoS Medicine*, 2(8): e124, 2005. doi: 10.1371/journal.pmed.0020124.

Ziwei Ji, Nayeon Lee, Rita Frieske, Tiezheng Yu, Dan Su, Yan Xu, Etsuko Ishii, Yejin Bang, Andrea Madotto, and Pascale Fung. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12):1–38, 2023. doi: 10.1145/3571730.

Qiao Jin, Yifan Yang, Qingyu Chen, and Zhiyong Lu. GeneGPT: Augmenting large language models with domain tools for improved access to biomedical information. *Bioinformatics*, 40 (2):btae075, 2024a. doi: 10.1093/bioinformatics/btae075.

Qiao Jin, Yifan Yang, Qingyu Chen, and Zhiyong Lu. GeneTuring tests for the field of computational genomics. *Nature Methods*, 21:768–778, 2024b. doi: 10.1038/s41592-024-02196-4.

Melissa J. Landrum, Jennifer M. Lee, Mark Benson, Garth R. Brown, Chen Chao, Shanmuga Chitipiralla, Baoshan Gu, Jennifer Hart, Douglas Hoffman, Wonhee Jang, Karen Karapetyan, Kenneth Katz, Chunlei Liu, Zenith Maddipatla, Adriana Malheiro, Kurt McDaniel, Michael Ovetsky, George Riley, George Zhou, J. Bradley Holmes, Brandi L. Kattman, and Donna R. Maglott. ClinVar: improving access to variant interpretations and supporting evidence. *Nucleic Acids Research*, 46(D1):D1062–D1067, 2018. doi: 10.1093/nar/gkx1153.

Arjun K. Manrai, Birgit H. Funke, Heidi L. Rehm, Morten S. Olesen, Barry A. Maron, Peter Szolovits, David M. Margulies, Joseph Loscalzo, and Isaac S. Kohane. Genetic misdiagnoses and the potential for health disparities. *New England Journal of Medicine*, 375(7):655–665, 2016. doi: 10.1056/NEJMsa1507092.

William McLaren, Laurent Gil, Sarah E. Hunt, Harpreet Singh Riat, Graham R. S. Ritchie, Anja Thormann, Paul Flicek, and Fiona Cunningham. The Ensembl variant effect predictor. *Genome Biology*, 17:122, 2016. doi: 10.1186/s13059-016-0974-4.

Kymberleigh A. Pagel, Rick Kim, Kyle Moad, Ben Busby, Lily Zheng, Collin Tokheim, Michael Ryan, and Rachel Karchin. Integrated Informatics Analysis of Cancer-Related Variants. *JCO Clinical Cancer Informatics*, 4:310–317, 2020. doi: 10.1200/CCI.19.00132. OpenCRAVAT. OakVar is the maintained successor.

Heidi L. Rehm, Jonathan S. Berg, Lisa D. Brooks, Carlos D. Bustamante, James P. Evans, Melissa J. Landrum, David H. Ledbetter, Donna R. Maglott, Christa Lese Martin, Robert L. Nussbaum, Sharon E. Plon, Erin M. Ramos, Stephen T. Sherry, and Michael S. Watson. ClinGen — the clinical genome resource. *New England Journal of Medicine*, 372(23):2235–2242, 2015. doi: 10.1056/NEJMsr1406261.

Sue Richards, Nazneen Aziz, Sherri Bale, David Bick, Soma Das, Julie Gastier-Foster, Wayne W. Grody, Madhuri Hegde, Elaine Lyon, Elaine Spector, Karl Voelkerding, and Heidi L. Rehm. Standards and guidelines for the interpretation of sequence variants: a joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology. *Genetics in Medicine*, 17(5):405–424, 2015. doi: 10.1038/gim.2015.30.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems*, volume 36, 2023.

Elliot Sollis, Annalisa Mosaku, Ala Abid, Annalisa Buniello, Maria Cerezo, Laurent Gil, Tudor Groza, Osman Güneş, Peggy Hall, James Hayhurst, Arwa Ibrahim, Yue Ji, Sajo John, Elizabeth John, G. Naamati, Michael Nicholson, A. Oyewole, E. Palumbo, Nigel W. Rayner, C. Sacramento, S. Trevanion, M. Zaharia, A. McMahon, L. Harris, H. Parkinson, and F. Cunningham. The NHGRI-EBI GWAS catalog: knowledgebase and deposition resource. *Nucleic Acids Research*, 51(D1):D977–D985, 2023. doi: 10.1093/nar/gkac1010.

Peter D. Stenson, Matthew Mort, Edward V. Ball, Mark Chapman, Katie Evans, Luisa Azevedo, Matthew Hayden, Sally Heber, and David N. Cooper. The Human Gene Mutation Database (HGMD): optimizing its use in a clinical diagnostic or research setting. *Human Genetics*, 139: 1197–1207, 2020. doi: 10.1007/s00439-020-02199-3.

Stephany Tandy-Connor, Jenna Guiltinan, Kate Krempely, Holly LaDuca, Patrick Reineke, Stephanie Gutierrez, Priscilla Gray, and Brigette Tippin Davis. False-positive results released by direct-to-consumer genetic tests highlight the importance of clinical confirmation testing for appropriate patient care. *Genetics in Medicine*, 20(12):1515–1521, 2018. doi: 10.1038/gim201838.

Kai Wang, Mingyao Li, and Hakon Hakonarson. ANNOVAR: functional annotation of genetic variants from high-throughput sequencing data. *Nucleic Acids Research*, 38(16):e164, 2010. doi: 10.1093/nar/gkq603.

Peng Wang and Kai Wang. PubMind: literature-based genetic variant extraction and functional annotation using large language models. *Nature Communications*, 2026. doi: 10.1038/s41467-026-76834-4. Article in Press.

Chih-Hsuan Wei, Alexis Allot, Kevin Riehle, Aleksandar Milosavljevic, and Zhiyong Lu. tmVar 3.0: an improved variant concept recognition and normalization tool. *Bioinformatics*, 38(18): 4449–4451, 2022. doi: 10.1093/bioinformatics/btac537.

Chih-Hsuan Wei, Alexis Allot, Robert Leaman, and Zhiyong Lu. PubTator 3.0: an AI-powered literature resource for unlocking biomedical knowledge. *Nucleic Acids Research*, 52(W1): W540–W546, 2024. doi: 10.1093/nar/gkae235.

Michelle Whirl-Carrillo, Ellen M. McDonagh, J. M. Hebert, Li Gong, Katrin Sangkuhl, Caroline F. Thorn, Russ B. Altman, and Teri E. Klein. Pharmacogenomics knowledge for personalized medicine. *Clinical Pharmacology & Therapeutics*, 92(4):414–417, 2012. doi: 10.1038/clpt.2012.96.

A Code and data availability

- **just-dna-format, just-dna-compiler, and just-dna-enricher**: `https://anonymous.4open.science/r/just-dna-compiler` (schema, reference compiler, and enricher).
- **just-module-creator**: `https://anonymous.4open.science/r/just-dna-registry` (MCP server, Claude Code and Codex plugin manifests, skills; MIT).
- **Just-DNA-Lite**: `https://anonymous.4open.science/r/just-dna-lite` (local VCF processing, annotation, and reporting; described in the companion paper [Anonymous, 2026]).
- The production catalog and staging polygon are live at URLs provided in the anonymized repositories.

B Research use only

Annotation modules summarize published association findings. They are not clinical-grade evidence. The authoring plugin does not make individual-level predictions and does not open a genome. Just-DNA-Lite and `just-prs` process sample data for research and educational use; their reports and scores are not diagnoses or medical advice. Language that implies causation, such as "causes" or "guarantees," is outside the scope of a module written with this plugin.

C Tools and command menu

Command menu reduction

The command redesign reduced the text placed in every session from 14,688 characters for twenty entries to 3,464 characters for seven entries, a 76% reduction. A test walks the links from those seven commands and confirms that every guide remains reachable. The smaller menu therefore removes repeated prompt content without making a stage inaccessible.

The title-as-quote observation

A measurement across 33 `studies.csv` files (44,342 rows) in the upstream repository found that 3,668 rows carried a `provenance_quote` that was the article's title verbatim. Since a title always appears in its own full text, the quote-verification check matched on all 3,668 rows while evidencing nothing about whether anyone located support for the row's claim. This observation led to the attribution requirement described in Section 3.3: a quote must name who located it in `StudyRow.curator` so that a reviewer can distinguish a machine-located passage from a human-read one.

Tools available to the assistant

The MCP tools are grouped by task: authoring, research, checks, enrichment, comparison, and registry work. The default configuration lists the full surface. An optional layered configuration initially lists the core authoring loop and a `toolbox` command that reveals the other groups. Tool search can reduce the listing further. These discovery options change what the agent sees in its context; they do not create a separate implementation of the workflow.

Registry writes require a token, with one necessary exception: `registry_register` creates the token and therefore cannot require one in advance. Tools whose cost depends on a large source corpus state that cost in their descriptions.

Literature and enrichment requests share a pacing gate, including the NCBI budget used by several lookups. The literature search fills a gap left by the enricher. The enricher can verify a PMID that an author already has, but it does not search for the paper. `literature_search` returns titles so the assistant can check identity. The existence of a PMID alone says nothing about whether it is the intended article. Supplementary-table retrieval extends the evidence path: the assistant can fetch, list, and inspect supplementary files attached to a cited paper, reaching the per-variant rows that GWAS main texts typically defer to their supplements.

How a person enters the workflow

A person usually knows what they want to accomplish, not which internal stage performs it. They may want to create a module, understand an inherited directory, find a paper, revise a published module, or decode an error message. The command menu uses those intentions as its entries.

Command	Purpose
`/create-module`	begin or continue creating a module
`/module-status`	inspect a directory and identify the next decision
`/module-revise`	begin another pass over an existing module
`/find-evidence`	find and read evidence for a row
`/module-publish`	rehearse and then publish
`/module-symptom`	explain a message from the toolchain
`/module-install-local`	install a compiled module without a catalog

The first version of the menu exposed twenty stage names. That forced a person to choose between terms such as `module-curate` and `module-enrich` before the system had explained either one. The seven commands now route to thirteen guides. A guide contains the procedure for one stage and is loaded by path when the assistant reaches that stage.

`/create-module` is therefore a router. It examines what the author has already provided and selects the next stage. If the request shows that the person first needs an explanation, the router loads the overview guide. The overview is not a menu command because a new author would not know its internal name.